ch 7 - Naive Bayes — Probability based technique

V1: Conditional Probability (Wikipedia of rolling 2 dice example)

$$P(A|B) = Pr(A=a \mid B=b)$$

(given; a: value; b: value)

A = random variable
B = " "

$$P(A|B) = \frac{P(A \cap B)}{P(B)} \quad \text{if } P(B) \neq 0 \rightarrow$$

— Definition of conditional Probability.

V2: Independent vs. Mutually exclusive events

Two events A and B are said to be independent if:

$$P(A|B) = P(A)$$
$$P(B|A) = P(B)$$

eg:
A: getting 6 in 1st Dice
B: " 3 " 2nd "

eg - $P(D_1 = 6 \mid D_2 = 3) = P(D_1 = 6)$
$P(D_2 = 3 \mid D_2 = 6) = P(D_2 = 3)$

A and B are said to be mutually exclusive events iff

If $P(A|B) = P(B|A) = 0$

then A & B are said to be mutually exclusive

when $P(A \cap B) = P(B \cap A) = 0$

eg- Event $A = D_1 = 6$ $P(A|B) = P(B|A) = 0$ ∵ B has already occurred A cannot happen.
" $B = D_1 = 3$

V3: Bayes Theorem with examples

$$P(A|B) = \frac{P(B|A)\,P(A)}{P(B)}, \text{ if } P(B) \neq 0$$

$P(A|B)$: posterior prob. $P(B|A)$: likelihood $P(A)$: prior $P(B)$: evidence

$$pos = \frac{l \cdot p}{e}$$ Not imp, to learn byheart

Proof: $P(A|B) = \frac{P(A \cap B)}{P(B)} = \frac{P(A, B)}{P(B)}$ (same)

In set theory, $A \cap B = B \cap A$

$$P(A|B) = \frac{P(B \cap A)}{P(B)} = \frac{P(B, A)}{P(B)}$$

(1)

$$P(B|A) = \frac{P(B \cap A)}{P(A)}$$

$$P(A) \times P(B|A) = P(B \cap A) \quad \text{——} (2)$$

Put 2 in 1

$$P(A|B) = \frac{P(A) \times P(B|A)}{P(B)}$$

(prior, likelihood, pos, evi)

Baye's Theo.

eg – 3 machines $M_1 = 20\%$ o/p Wikipedia
$M_2 = 30\%$ " article
$M_3 = 50\%$ " (A complicated example)

Item chosen at random found to be defective. Prob. that it belongs to 3rd machine?

Fraction of defective items for :-
$M_1 = 5\%$
$M_2 = 3\%$
$M_3 = 1\%$

Ans $P(M_3|D) = ?$ $\frac{P(D|M_3)P(M_3)}{P(D)}$ (D = defective)

$P(M_1) = 0.2$

$P(M_2) = 0.3$

$P(M_3) = 0.5$

$P(D|M_1) = 0.05$

$P(D|M_2) = 0.03$

$P(D|M_3) = 0.01$

$$P(D) = \sum_{i=1}^{3} P(D|M_i) \times P(M_i)$$

$$= (0.2 \times 0.05) + (0.3 \times 0.03) + (0.5 \times 0.01)$$

$$= 0.024 \quad (2.4\% \text{ defective item})$$

$$P(M_3|D) = \frac{0.01 \times 0.5}{0.024} = \boxed{\frac{5}{24}}$$

Exercise problems → Practise Bayes Theorem Questions for interview.

classifier

V5: Naive Bayes Algorithm

Consider a data point with n-features and K classes $(C_1, C_2 \cdots C_K)$ ← classes

If $K=2$, then binary classification

$x = \langle x_1, x_2 \cdots x_n \rangle$ ← features

Task: Given x (all n features), we have to predict its class label.

ie: $\boxed{P(C_k|x) \; \forall \; k \in [1, K]}$

whichever is the highest we'll select that as the class label.

$$P(C_k|x) = \frac{P(x|C_k) \cdot P(C_k)}{P(x)}$$

$$\Rightarrow P(x \cap C_k) = P(x, C_k)$$

(only numerator)

Denominator constant for all class.
So, only numerator matters.

So, $\boxed{P(C_k|x) \propto P(C_k, x)}$ Joint Probability Model.

not exactly equal to but proportional

General formula of conditional prob

$P(A,B) = P(A|B) * P(B)$

Same applied below.

$P(C_k, x) = P(C_k, x_1, x_2, x_3 \ldots x_n)$

Chain rule of conditional probability:

$P(C_k, x_1, x_2 \ldots x_n) = P(\underbrace{x_1}_{A}, \underbrace{x_2, x_3 \ldots x_n, C_k}_{B})$

$= P(\underbrace{x_1}_{A} | \underbrace{x_2, x_3 \ldots x_n, C_k}_{B}) * P(x_2, x_3 \ldots x_n, C_k)$

$= P(x_1 | x_2, x_3 \ldots x_n, C_k) * P(x_2 | x_3, x_4 \ldots C_k) * P(x_3, x_4, \ldots x_n, C_k)$

$= P(x_1 | x_2, x_3 \ldots x_n, C_k) * P(x_2 | x_3, x_4 = x_n, C_k) * P(x_{n-1} | x_n, C_k) * P(x_n | C_k) \cdot P(C_k)$

It is difficult to calculate all the values in datapoint

Assuming that each feature x_i is conditionally independent of every other feature,

$P(A|B,C) = P(A|C)$ { if A & B are independent }

$P[C_k, x_1, x_2 \ldots x_n] = P(x_1|C_k)\, P(x_2|C_k) \ldots P(x_n|C_k) \cdot P(C_k)$

$= P(C_k) \prod_{i=1}^{n} P(x_i|C_k)$

So, $P(C_k | x_1, x_2 \ldots x_n) \propto P(C_k) \prod_{i=1}^{n} P(x_i|C_k)$

$= \frac{1}{Z} P(C_k) \prod_{i=1}^{n} P(x_i|C_k)$

Note: $Z = P(x)$ denominator term which was

ignored and made numerator proportional to.

$$\hat{y} = \underset{\text{where } k \in (1,K)}{\operatorname{argmax}} \; p(C_k) \prod_{i=1}^{n} P(x_i | C_k)$$

Maximum a posteriori rule

Train :-
Time = $O(n \cdot d \cdot c)$ (n: datpoints, d: dimension or features, c: classes) } Training phase.
Space = $O(d * c)$ (d: dimension or features, c: classes)

Test :-
Time = $O(d * c)$. } Since only testing point and both class prob.
if d and c is small

NB is much better than KNN esp. in terms of Space Complexity.

V7: Naive Bayes on Text data

eg- SPAM filter: email — SPAM. / NOT SPAM. } classification

eg Review — +ve / -ve

Task:- $P(y_q=1 | text_q)$ / $P(y_q=0 | text_q)$ } Binary classification

Text Preprocessing — Stopwords
Stemming
n_gram

Using binary BOW representation

After preprocessing =>

$text \rightarrow \{w_1, w_2, w_3 \ldots w_d\}$

$P(y=1 | w_1, w_2, \ldots w_d)$ — BOW features

$P(y=1 | text_q) \propto P(y=1) * P(w_1 | y=1) * P(w_2 | y=1) * \ldots * P(w_d | y=1)$

$P(y=1 | text) \propto \underbrace{P(y=1)}_{\text{prior}} \prod_{i=1}^{d} \underbrace{P(w_i | y=1)}_{\text{likelihood}}$

Similarly, $P(y=0 | text) \propto \underbrace{P(y=0)}_{\text{prior}} \prod_{i=1}^{d} \underbrace{(w_i | y=0)}_{\text{likelihood}}$

Calculating class priors:-

$P(y=1) = \frac{\#\text{ of Train points with } y=1}{\text{Total train points}}$

$P(y=0) = \frac{\#\text{ of Train points with } y=0}{\text{Total train points}}$

Calculating likelihood:-

For $y=1$

$$P(w_i|y=1) = \frac{\#\text{ of data points which contain word } w_i \text{ \&\& } y=1}{\#\text{ of data points with } y=1}$$

Similarly for $y=0$

$$P(w_i|y=0) = \frac{\#\text{ of data points which contain word } w_i \text{ \&\& } y=0}{\#\text{ of data points with } y=0}$$

All the above formulas need to be applied on train data only.

NB is used as a baseline model for text classification.

We can use any performance metric for benchmark.

V8: Laplace/Additive Smoothing

At the end of training phase, you have your likelihoods:-

$P(y=1)$ $P(y=0)$ — class priors

$P(w_1|y=1)$ $P(w_1|y=0)$

$P(w_2|y=1)$ $P(w_2|y=0)$ } likelihoods

$P(w_m|y=1)$ $P(w_m|y=0)$

In Test phase:- (present in train)

Say test = $\{w_1, w_2, w_3, w'\}$ ← new word

$$P(y=1 \mid \text{text}) = P(y=1) \cdot P(w_1 \mid y=1) \cdot P(w_2 \mid y=1) \cdot P(w_3 \mid y=1) \cdot P(w' \mid y=1)$$

If we completely ignore $P(w' \mid y=1)$ then it will be same as assuming

$P(w' \mid y=0) = 1$ }
$P(w' \mid y=1) = 1$ } which is absurd.

If we go by definition, $P(w' \mid y=1)$

$$= \frac{\#\text{ of Train data points where } w' \text{ occurs } \&\& \ y=1}{\#\text{ of Train data points with } y=1.}$$

$$= \frac{0}{n_1} = 0$$

This will make the entire probability 0.

So, we do laplace Smoothing, also called as additive smoothing. (not laplacian Smoothing)

$$P(w' \mid y=1) = \frac{0+\alpha}{n_1 + \alpha k}, \quad k = \text{no. of distinct values that feature } w' \text{ can take.}$$

Here, since we are using Binary BoW representation, $k=2$.

α can be any numeric value, typically taken as 1.

Let $n_1 = 100$

$$P(w' \mid y=1) = \frac{0+\alpha}{100+2\alpha}$$

(Also you get rid of the division by zero problem)

Case 1: $\alpha = 1$. $P(w' \mid y=1) = \frac{1}{102}$

Case 2: $\alpha = 10000$ $P(w' \mid y=1) = \frac{10000}{20100} \approx \frac{1}{2}$.

So $P(w' \mid y=1) \geq P(w' \mid y=0) = \frac{1}{2}$

Laplace Smoothing when applied is applied to all the words, not to only the words not present in training

$$P(w_i \mid y=1) = \frac{\#\text{ of data points with } w_i \text{ \& } y=1 + \alpha}{\#\text{ of data points with } y=1 + \alpha k}$$

uniform distribution is stable distribution
As $\alpha \uparrow$, we are moving closer to $\frac{1}{2}$.

As $\alpha \uparrow$, it moves the likelihood probabilities to uniform distribution

So, if we have a likelihood prob. with small $\frac{\text{num}}{\text{deno}}$, then we have less confidence in the ratio. So, if we have just large enough α, it'll smoothen its prob. value towards uniform distribution.

when $\alpha = 1$, it is called add-one smoothing.

V9: Log probabilities for numerical stability

Since all the probabilities are b/w 0 & 1, & also text data are having many features, continued product may cause underflow of data. $(0.2 \times 0.1 \times 0.2 \times 0.1) = 0.0004$

Python stores 16 precision decimal pts eg- numerical underflow.

So, we take log of all the class priors & likelihood probabilities and add them together $\log(P(y=1|w_1 \cdots w_d)) = P(y=1) + \prod_{i=1}^{d} P(w_i|y=1)$

$\log(P(y=0|w_1 \cdots w_d)) = P(y=0) + \prod_{i=1}^{d} P(w_i|y=0)$

The class having maximum value is selected.

Since log is monotonically increasing it is suitable for this purpose.

→ we have $\log(a \times b) = \log a + \log b$.

So, it becomes $\log P(y=1) + \sum_{i=1}^{d} \log(P(w_i|y=1))$ ∀y for $P=0$.

Date Page

e 2: α is v. large

$\alpha = 10,000.$

$$P(w_0 | y=1) = \frac{2}{1000} + \frac{10000}{20000} \approx \frac{1}{2}$$

K

K becomes v. large, $P(w_0 | y=1) \approx P(w_0 | y=0)$

$\approx \frac{1}{2}.$

$$\Rightarrow P(y_q = 1 | x_q) \approx P(y_q = 0 | x_q) \approx \frac{1}{2}$$

Underfitting

Like in KNN ($K=n$) $x_q \rightarrow$ same class used

Here also it will be given same class.

$\alpha =$ v. large

$$P(y=1 | \underbrace{w_1, w_2 \ldots w_d}_{x_q}) = \underbrace{P(y=1)}_{n_1/n} * \prod_{i=1}^{d} P(w_0 | y=1)$$

$1/2$

$$P(y=0 | w_1, w_2 \ldots w_d) = \underbrace{P(y=0)}_{n_2/n} * \prod_{i=1}^{d} P(w_0 | y=0)$$

In KNN $n_1 = +ve$, $n_2 = -ve$

$n_1 > n_2$

$\frac{1}{2}$

It was underfitting.

Why in NB, both ... down to bottom, $n - [n_1 + n_2 - w]$

Case 1 $\alpha = 0$

$P(w_i | y=1) = \frac{\#\text{ of Train data pts } w_i \text{ occurs } \& \; y=1}{\#\text{ of Train pts with } y=1}$

$= \frac{2}{1000}$ — only 2 times — 1000 — +ve data pts.

Let $n = 2K$
+ve — 1K
-ve — 1K

So the word which occurs 2 times is very rare.

overfitting of words that are rare $P(w_i | y=1)$

If the train data changes result in model change dramatically.
Since w_i occurs only 2 out of 2K cases (1K +ve, 1K -ve)
If I change

Small change in my D_{train}, removing those 2 texts (pts that contain w_i),

$$P(w_i | y=1) = \frac{2}{1000} \text{ to } \frac{0}{1000}$$

Small change in data causes large change model :: prob. went from 2/1000 to 0/1000.

Small change in D_{train} results in large change in the model - high variance

V10 : Bias-Variance Tradeoff

High bias = Underfitting
High Variance = Overfitting

Only hyperparameter = α (Laplace smoot
determines whether overfitting or underfitting.
High α - Smoothens out likelihood probabilities towards uniform distribution
According to previous page prob. formula

Case 1: $\alpha = 0$ with small change in training data likelihood probability changes a lot. $P(w_i|y=1) = \frac{\#\text{ of Train data pts. } w_i \text{ occurs } \& y=1}{\#\text{ of Train pts with } y=1}$

$\Rightarrow$ High Variance = overfitting.

Case 2: α = large, for all w_i ; $P(w_i|y=1) \approx \frac{1}{2}$
$\alpha = 10000 \Rightarrow P(w_i|y=0) = \frac{1}{2}$
if α is significantly high.

So, Underfitting. High bias

α of optimum value must be calculated using Cross Validation.

Hyperparameter α controls bias-variance tradeoff in Naive Bayes.

VII: Feature Importance and Interpretability

Naive Bayes

words with high value of :-

$\forall w_i$,

$P(w_i | y=1)$ → important words / features in determining that point belongs to +ve class.

So, feature importance can be directly obtained from model, unlike FFS in KNN

{ +ve feature : Find words (w_i) with highest value of $P(w_i | y=1)$ (very imp. words in determining that x_q belongs to +ve class)

{ -ve feature : Find words (w_i) with highest value of $P(w_i | y=0)$ (very imp. word / feature in terms of -ve class.

Interpretability

$x_q \longrightarrow y_q = 1$

$\{w_1, w_2, \ldots w_d\}$ | $y_q = 1$ ∵ x_q contains w_3, w_6, w_{10} which have high value of $P(w_3|y=1)$, $P(w_6|y=1)$, $P(w_{10}|y=1)$ and also $y_q \neq 0$ ∵ w_8, w_{15}, w_{20} have $P(w_8|y=0)$, $P(w_{15}|y=0)$, $P(w_{20}|y=0)$ are small.

details

classmate
Date
Page

$\forall w_i$, we have $P(w_i|y=0)$ & $P(w_i|y=$

So,

w_i	$P(w_i\|y=1)$	$P(w_i\|y=0)$
w_1		
w_2		
:		
:		

1) Sort w_i's based on $P(w_i|y=1)$ d
order.

Top words $P(w_i|y=1)$

V12: Imbalanced data

Let n_1 be +ve class and n_2 be -ve class. Let $n_1 >> n_2$ or $n_2 >> n_1$. This indicates it is an imbalanced dataset. The train dataset has 90% +ve class datapoints and 10% negative class datapoints.

Say $\frac{n_1}{n} = 0.9$ and $\frac{n_2}{n} = 0.1$

$$P(y=1 \mid w_1, w_2 \cdots w_d) = \underbrace{P(y=1)}_{n_1/n} \prod_{i=1}^{d} P(w_i \mid y=1)$$

// equal let

$$P(y=0 \mid w_1, w_2, \cdots w_d) = \underbrace{P(y=0)}_{n_2/n} \prod_{i=1}^{d} P(w_i \mid y=0)$$

If the likelihoods are almost the same then the majority (n_1) class has an advantage due to high class prior.

Solution to this problem:-

1) Upsampling or downsampling
So, $P(y=1) = P(y=0) = \frac{1}{2}$.
We only compare probabilities
2) Simply drop the probabilities terms.
3) Use modified NB (which is mostly in research papers and not practically used).

Another problem with imbalanced data

+)ve $n_1 = 900 \longrightarrow P(w_i | y=1) = \frac{\boxed{\quad} + (0.900)}{900}$

-)ve $n_2 = 100 \longrightarrow P(w_0 | y=0) = \frac{\boxed{\quad} + (0.100)}{100}$

So, minority class have small numerator and denominator

When we apply laplace smoothing, the impact of α is more on minority than on majority

Let $\alpha = 10$

Ex

Minority	Majority
$\frac{2}{100} \approx 2\%$	$\frac{18}{900} \approx 2\%$
$\frac{2+10}{100+20}$	$\frac{18+10}{900+20}$
$= 10\%$	$= 3.04\%$
$= 0.1$	

very huge difference in minority but not so in majority.

Date: SMTWTFS

Hack

Use one α for +ve class another for -ve class.

Better solution is always to use upsampling or downsampling.

V13: Outliers

To solve the problem of outliers :-

1) If a word w_j occurs fewer times then just ignore that word.

If w_j occurs 10 times, keep $T = 10$. (#of times word occurs in train data)

$w_j \notin \{w_1, w_2, \ldots w_m\}$ occurs very few times in D_{train}

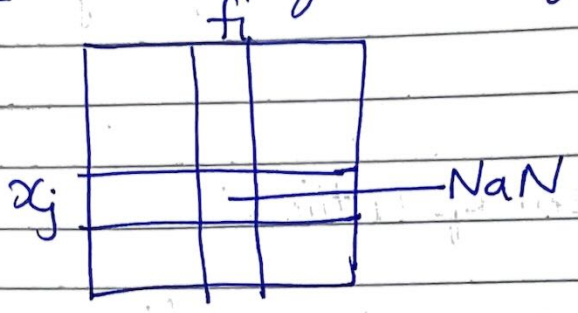
or simply discard that word.

2) Laplace Smoothing α should be introduced so outliers in train as well as (unseen) at test time is taken care by Laplace Smoothing.

VI4: Missing Values

Case 1: In text data no problem of missing data. Either a word occurs or it doesn't.

Case 2: In Categorical features



$f_i \in \{a_1, a_2, a_3\}$

Hack: Assume NaN is also one category.

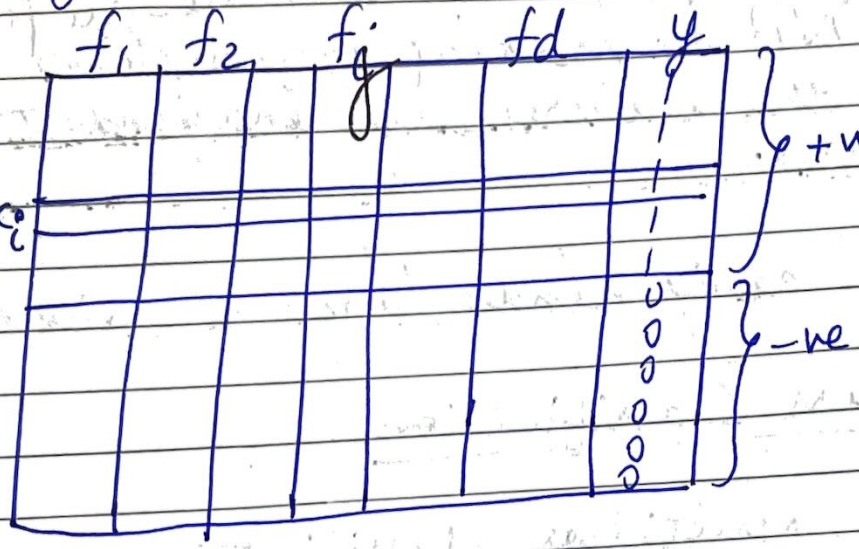
$f_i \in \{a_1, a_2, a_3, NaN\}$

Case 3: Numerical features

We apply standard feature imputation like mean, median, mode.

V15: Handling Numerical features (Gaussian Naive Bayes):-

Say we have a dataset like this :-



f_j : real valued

$$P(y_i = 1 \mid x_{i1}, x_{i2}, x_{i3} \ldots x_{id})$$

$$= P(y_i = 1) \prod_{j=1}^{d} P(x_{ij} \mid y_i = 1)$$

/ class prior

$P(x_{ij} \mid y = 1)$ Take $D_{train}^{+ve} = D'$

$P(x_{ij} \mid D')$ Calculate mean & variance of j^{th} feature say μ & σ

Plot PDF, you get PDF of f_j for D' after assuming feature f_j to be a

Gaussian distribution of $N(\mu, \sigma)$

This plot and concept is also called Gaussian Naive Bayes. We can get ~~class~~ probs from pdf of the plot for both the +ve & −ve pts. ~~Gaussian~~ NB is making assumption that all the features are independent of each other.

Also NB can easily handle large dimensions.

V16: Multiclass Classification

It can do multiclass classification.

$P(y_i = 1 \mid w_1, w_2 \ldots w_d)$
$P(y_i = 0 \mid w_1, w_2 \ldots w_d)$
$P(y_i = 2 \mid w_1, w_2 \ldots w_d)$
.
.
$P(y_i = C \mid w_1, w_2 \ldots w_d)$

} compare and get largest proba.

1K +ve
1K −ve

C | class

It is ~~def~~ designed to do multi-class classification. Binary is a special case.

V17: Similarity or Distance matrix

Can NB do classification given distance/similarity matrix?

Cannot use distance/similarity matrix. because you have to compute proba. So you need the exact feature value which is impossible when you use distance.
KNN can do.

V18: Large Dimensionality

Can NB handle large dimension data. It is used in text data which is large. But you need to use log probabilities. So that you don't have numerical underflow/stability issues.

V19: Best and Worst Cases

1) Conditional Independence of features given any class label. If it is true, NB performs very well. But if it is false, NB performance deteriorates.

2) Text classification - NB used as the baseline model for SPAM filter, review polarity.

3) Categorical features - If you have real valued features, seldom used. but very good for categorical data

~~4) Real valued features~~

4) Interpretability and feature Importance is great. We can reason as to why the model gave a certain output

Runtime Complexity is low.
Train time " is low
Runtime Space complexity is low

5) we only have to save prior & likelihood probs.
Can easily overfit unless laplace smoothing done with proper α from cross validation.